Repositorio de Ejercicios Resueltos

Semana 1 — Aplicando Polimorfismo en Modelado de Clases (Sobrescritura y Sobrecarga)

PRY2203 · Desarrollo Orientado a Objetos II · Experiencia 1

Índice

Introducción **2**

Ejercicio 1 — Polimorfismo por sobrescritura: bonos de personal logístico **3**

Ejercicio 2 — Sobrecarga de métodos: disponibilidad de repartidores **5**

Ejercicio 3 — Caso guía: Sistema de Validación de Pasajes **7**

Ejercicio 4 — Polimorfismo combinado: reservas de una academia deportiva **9**

Ejercicio 5 — Extensión del sistema de reservas (principio Abierto/Cerrado) **13**

Ejercicio 6 — Sobrescritura: figuras geométricas y cálculo de área **15**

Ejercicio 7 — Sobrecarga de métodos: clase Impresora **17**

Ejercicio 8 — Polimorfismo combinado: sistema de préstamos de una biblioteca **19**

Ejercicio 9 — Análisis de código: identificar sobrescritura y sobrecarga **22**

Ejercicio 10 — Extensión del sistema de biblioteca (principio Abierto/Cerrado) **23**

Introducción

Este repositorio reúne **diez ejercicios resueltos** para reforzar los tres conceptos centrales de la Semana 1 de la Experiencia 1 (*Desarrollo Orientado a Objetos II*): **polimorfismo, sobrescritura de métodos y sobrecarga de métodos**.

Los ejercicios están construidos tomando como referencia la **guía de aprendizaje** (sistema *SistemaValidacionPasajes* con las clases `Usuario`, `Estudiante`, `Adulto`, `TerceraEdad` y `Validador`) y proponen **escenarios de negocio propios y originales** —personal logístico, disponibilidad de repartidores, reservas deportivas, figuras geométricas, impresión de documentos, préstamos de biblioteca y análisis de código— para practicar los mismos principios sin repetir el caso de la tarea evaluada (SpeedFast).

Cada ejercicio incluye: **enunciado**, **análisis de la solución**, **código Java completo** y **salida de consola esperada**, para que puedas comparar tu propia resolución con la propuesta.

Índice de ejercicios:

1. Polimorfismo por sobrescritura — bonos de personal logístico
2. Sobrecarga de métodos — disponibilidad de repartidores
3. Caso guía — Sistema de Validación de Pasajes (modelo de referencia)
4. Polimorfismo combinado — reservas de una academia deportiva
5. Extensión del sistema de reservas (principio Abierto/Cerrado)
6. Sobrescritura — figuras geométricas y cálculo de área
7. Sobrecarga de métodos — clase Impresora
8. Polimorfismo combinado — sistema de préstamos de una biblioteca
9. Análisis de código — identificar sobrescritura y sobrecarga
10. Extensión del sistema de biblioteca (principio Abierto/Cerrado)

Ejercicio 1 — Polimorfismo por sobrescritura: bonos de personal logístico

Enunciado

Una empresa de logística tiene tres tipos de trabajadores: **Repartidor**, **Supervisor** y **Administrativo**. Todos comparten un atributo `nombre` y deben poder calcular su bono mensual mediante un método `calcularBono()`, pero cada tipo de trabajador lo calcula con una lógica distinta:

- `Repartidor`: bono fijo de $20.000 por "entregas completadas sin incidentes".
- `Supervisor`: bono fijo de $35.000 por "cumplimiento de metas de equipo".
- `Administrativo`: bono fijo de $15.000 por "apoyo operativo".

Se pide:

1. Crear una clase base `Empleado` con el atributo `nombre` (encapsulado) y un método `calcularBono()` con un mensaje genérico.
2. Crear las tres subclases, cada una **sobrescribiendo** `calcularBono()` con su propia lógica.
3. Desde una clase `Main`, crear un arreglo de tipo `Empleado[]` con un objeto de cada subclase y recorrerlo invocando `calcularBono()`, demostrando el polimorfismo en tiempo de ejecución.

Análisis de la solución

Este ejercicio aplica **polimorfismo por sobrescritura** (el mismo concepto usado en la guía con `Usuario → Estudiante/Adulto/TerceraEdad`). La clave está en que el arreglo se declara como `Empleado[]`, pero en tiempo de ejecución cada posición ejecuta el método de su **clase real**, no el de la clase base. Esto evita usar bloques `if/else` para diferenciar el tipo de trabajador.

Resolución

```java
// Empleado.java
public class Empleado {
    private String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }

    public void calcularBono() {
        System.out.println(nombre + ": bono general no definido.");
    }
}
```

```java
// Repartidor.java
public class Repartidor extends Empleado {
    public Repartidor(String nombre) {
        super(nombre);
    }

    @Override
    public void calcularBono() {
```

```
        System.out.println(getNombre() + ": bono de $20.000 por entregas sin incidentes.");
    }
}
```

```
// Supervisor.java
public class Supervisor extends Empleado {
    public Supervisor(String nombre) {
        super(nombre);
    }

    @Override
    public void calcularBono() {
        System.out.println(getNombre() + ": bono de $35.000 por cumplimiento de metas de equ
    }
}
```

```
// Administrativo.java
public class Administrativo extends Empleado {
    public Administrativo(String nombre) {
        super(nombre);
    }

    @Override
    public void calcularBono() {
        System.out.println(getNombre() + ": bono de $15.000 por apoyo operativo.");
    }
}
```

```
// Main.java
public class Main {
    public static void main(String[] args) {
        Empleado[] equipo = {
            new Repartidor("Juan Pérez"),
            new Supervisor("Camila Soto"),
            new Administrativo("Luis Díaz")
        };

        for (Empleado e : equipo) {
            e.calcularBono();
        }
    }
}
```

Salida de consola esperada

```
Juan Pérez: bono de $20.000 por entregas sin incidentes.
Camila Soto: bono de $35.000 por cumplimiento de metas de equipo.
Luis Díaz: bono de $15.000 por apoyo operativo.
```

Ejercicio 2 — Sobrecarga de métodos: disponibilidad de repartidores

Enunciado

Se necesita una clase `GestorDisponibilidad` que permita consultar la disponibilidad de repartidores de tres formas distintas, según la información disponible al momento de la consulta:

1. Sin parámetros: hace una verificación general.
2. Con el nombre de la **zona**: verifica disponibilidad solo en esa zona.
3. Con la **zona** y un `boolean` que indica si se requiere **turno noche**.

Se pide implementar los tres métodos sobrecargados `verificarDisponibilidad()` y demostrarlos desde `Main`.

Análisis de la solución

A diferencia del Ejercicio 1, aquí **no hay herencia**: los tres métodos conviven en la **misma clase**, comparten el nombre `verificarDisponibilidad`, y se diferencian por la **lista de parámetros** (cantidad y tipo). Esta es la firma distintiva de la **sobrecarga** (polimorfismo en tiempo de compilación): el compilador decide qué versión ejecutar según los argumentos que se le entregan en cada llamada.

Resolución

```java
// GestorDisponibilidad.java
public class GestorDisponibilidad {

    // Versión 1: sin parámetros
    public void verificarDisponibilidad() {
        System.out.println("Verificación general de disponibilidad en toda la flota.");
    }

    // Versión 2: con zona
    public void verificarDisponibilidad(String zona) {
        System.out.println("Verificando disponibilidad en la zona: " + zona);
    }

    // Versión 3: con zona y turno noche
    public void verificarDisponibilidad(String zona, boolean turnoNoche) {
        if (turnoNoche) {
            System.out.println("Buscando repartidores de turno noche en " + zona + "...");
        } else {
            System.out.println("Buscando repartidores de turno día en " + zona + "...");
        }
    }
}
```

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        GestorDisponibilidad gestor = new GestorDisponibilidad();

        gestor.verificarDisponibilidad();
        gestor.verificarDisponibilidad("Providencia");
```

```
        gestor.verificarDisponibilidad("Providencia", true);
    }
}
```

Salida de consola esperada

```
Verificación general de disponibilidad en toda la flota.
Verificando disponibilidad en la zona: Providencia
Buscando repartidores de turno noche en Providencia...
```

Ejercicio 3 — Caso guía: Sistema de Validación de Pasajes

Enunciado

(Ejercicio base de la guía de aprendizaje, resuelto como modelo de referencia.)

Se debe modelar un sistema de validación de pasajes para transporte público. Existe una clase `Usuario` con un método `validarPasaje()`. De ella derivan `Estudiante`, `Adulto` y `TerceraEdad`, cada una con su propia tarifa. Además, se debe crear una clase `Validador` independiente que **sobrecargue** `validarPasaje()` en tres versiones: sin datos, con el tipo de usuario, y con tipo de usuario más edad (aplicando descuento si corresponde a tercera edad con 60 años o más).

Análisis de la solución

Este ejercicio combina en un mismo sistema **ambos** tipos de polimorfismo:

- **Sobrescritura** entre `Usuario` y sus subclases (`Estudiante`, `Adulto`, `TerceraEdad`), resuelta en tiempo de ejecución según el tipo real del objeto.
- **Sobrecarga** dentro de la clase `Validador`, resuelta en tiempo de compilación según los argumentos entregados.

Resolución

```java
// Usuario.java
public class Usuario {
    public void validarPasaje() {
        System.out.println("Validación general de pasaje.");
    }
}
```

```java
// Estudiante.java
public class Estudiante extends Usuario {
    @Override
    public void validarPasaje() {
        System.out.println("Tarifa estudiante aplicada.");
    }
}
```

```java
// Adulto.java
public class Adulto extends Usuario {
    @Override
    public void validarPasaje() {
        System.out.println("Tarifa adulto aplicada.");
    }
}
```

```java
// TerceraEdad.java
public class TerceraEdad extends Usuario {
    @Override
    public void validarPasaje() {
        System.out.println("Tarifa tercera edad aplicada.");
    }
}
```

```java
// Validador.java
public class Validador {
```

```java
    // Versión 1: sin parámetros
    public void validarPasaje() {
        System.out.println("Validación estándar sin datos.");
    }

    // Versión 2: con tipo de usuario
    public void validarPasaje(String tipoUsuario) {
        System.out.println("Validación para tipo: " + tipoUsuario);
    }

    // Versión 3: con tipo de usuario y edad
    public void validarPasaje(String tipoUsuario, int edad) {
        if ("Tercera Edad".equalsIgnoreCase(tipoUsuario) && edad >= 60) {
            System.out.println("Descuento aplicado por edad avanzada.");
        } else {
            System.out.println("Validación para " + tipoUsuario + ", edad: " + edad);
        }
    }
}
```

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        Usuario[] usuarios = {
            new Estudiante(),
            new Adulto(),
            new TerceraEdad()
        };

        for (Usuario u : usuarios) {
            u.validarPasaje();
        }

        Validador validador = new Validador();
        System.out.println("\n=== Validación con sobrecarga ===");
        validador.validarPasaje();
        validador.validarPasaje("Estudiante");
        validador.validarPasaje("Tercera Edad", 65);
    }
}
```

Salida de consola esperada

```
Tarifa estudiante aplicada.
Tarifa adulto aplicada.
Tarifa tercera edad aplicada.

=== Validación con sobrecarga ===
Validación estándar sin datos.
Validación para tipo: Estudiante
Descuento aplicado por edad avanzada.
```

Ejercicio 4 — Polimorfismo combinado: reservas de una academia deportiva

Enunciado

Una academia deportiva ofrece tres modalidades de reserva: **Clase grupal**, **Entrenamiento personal** y **Uso libre de cancha**. Cada modalidad confirma la reserva de forma distinta:

- `ReservaClaseGrupal`: valida que existan **cupos disponibles** en la clase.
- `ReservaEntrenamientoPersonal`: valida que el **entrenador solicitado esté libre** en el horario.
- `ReservaCanchaLibre`: valida que la **cancha no esté ocupada** en el horario solicitado.

Se pide diseñar el sistema aplicando polimorfismo, en dos fases:

Fase 1 — Comportamiento diferenciado (sobrescritura): - Crear una clase base `Reserva` con los atributos encapsulados `idReserva`, `horario` y `tipoReserva`, y un constructor completo. - Crear el método `confirmarReserva()` en `Reserva` con un mensaje genérico. - Crear las subclases `ReservaClaseGrupal`, `ReservaEntrenamientoPersonal` y `ReservaCanchaLibre`, cada una **sobrescribiendo** `confirmarReserva()` con la validación correspondiente a su modalidad.

Fase 2 — Flexibilidad de confirmación (sobrecarga): - En cada subclase, agregar una **segunda versión** `confirmarReserva(String nombreResponsable)` (sobrecarga) que reciba el nombre de la persona que confirma la reserva (instructor, entrenador o encargado de cancha) e imprima un mensaje que incluya tanto la validación propia de la modalidad como la confirmación explícita a nombre de esa persona.

Prueba de integración: desde `Main`, instanciar al menos un objeto de cada subclase e invocar ambas versiones del método para demostrar el polimorfismo.

Análisis de la solución

Este ejercicio integra los dos mecanismos trabajados durante la semana en un problema de negocio distinto al de transporte:

- **Sobrescritura (Fase 1):** `Reserva.confirmarReserva()` se redefine en cada subclase con la **misma firma**, pero **distinto comportamiento**, evitando condicionales `if/else` según `tipoReserva` dentro de una única clase.
- **Sobrecarga (Fase 2):** dentro de cada subclase conviven `confirmarReserva()` (sobrescrito, sin parámetros) y `confirmarReserva(String nombreResponsable)` (nuevo, con parámetro). El compilador elige la versión correcta según los argumentos de cada llamada.

Esta arquitectura respeta el **principio abierto/cerrado** (“abierto a la extensión, cerrado a la modificación”), que es justamente lo que la hace escalable ante nuevas modalidades de reserva.

Resolución

```java
// Reserva.java
public class Reserva {
    private String idReserva;
    private String horario;
    private String tipoReserva;

    public Reserva(String idReserva, String horario, String tipoReserva) {
        this.idReserva = idReserva;
        this.horario = horario;
        this.tipoReserva = tipoReserva;
```

```
    }

    public String getIdReserva() {
        return idReserva;
    }

    public String getHorario() {
        return horario;
    }

    public String getTipoReserva() {
        return tipoReserva;
    }

    // Versión base (sin parámetros) — será sobrescrita en cada subclase
    public void confirmarReserva() {
        System.out.println("Confirmando reserva...");
        System.out.println("-> Validación genérica de reserva.");
    }
}
```

```
// ReservaClaseGrupal.java
public class ReservaClaseGrupal extends Reserva {

    public ReservaClaseGrupal(String idReserva, String horario) {
        super(idReserva, horario, "Clase grupal");
    }

    // Sobrescritura: validación de cupos disponibles
    @Override
    public void confirmarReserva() {
        System.out.println("[Reserva Clase Grupal]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando cupos disponibles... OK");
    }

    // Sobrecarga: versión con nombre del responsable
    public void confirmarReserva(String nombreResponsable) {
        System.out.println("[Reserva Clase Grupal]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando cupos disponibles... OK");
        System.out.println("-> Reserva confirmada por " + nombreResponsable);
    }
}
```

```
// ReservaEntrenamientoPersonal.java
public class ReservaEntrenamientoPersonal extends Reserva {

    public ReservaEntrenamientoPersonal(String idReserva, String horario) {
        super(idReserva, horario, "Entrenamiento personal");
    }

    // Sobrescritura: validación de disponibilidad del entrenador
    @Override
    public void confirmarReserva() {
```

```
        System.out.println("[Reserva Entrenamiento Personal]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando disponibilidad del entrenador... OK");
    }

    // Sobrecarga: versión con nombre del responsable
    public void confirmarReserva(String nombreResponsable) {
        System.out.println("[Reserva Entrenamiento Personal]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando disponibilidad del entrenador... OK");
        System.out.println("-> Reserva confirmada por " + nombreResponsable);
    }
}
```

```
// ReservaCanchaLibre.java
public class ReservaCanchaLibre extends Reserva {

    public ReservaCanchaLibre(String idReserva, String horario) {
        super(idReserva, horario, "Cancha libre");
    }

    // Sobrescritura: validación de disponibilidad de la cancha
    @Override
    public void confirmarReserva() {
        System.out.println("[Reserva Cancha Libre]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando disponibilidad de la cancha... OK");
    }

    // Sobrecarga: versión con nombre del responsable
    public void confirmarReserva(String nombreResponsable) {
        System.out.println("[Reserva Cancha Libre]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando disponibilidad de la cancha... OK");
        System.out.println("-> Reserva confirmada por " + nombreResponsable);
    }
}
```

```
// Main.java
public class Main {
    public static void main(String[] args) {
        Reserva[] reservas = {
            new ReservaClaseGrupal("R-001", "09:00"),
            new ReservaEntrenamientoPersonal("R-002", "11:00"),
            new ReservaCanchaLibre("R-003", "18:00")
        };

        // Comportamiento base (versión sobrescrita, sin parámetros)
        System.out.println("=== Invocando comportamiento base ===\n");
        for (Reserva r : reservas) {
            r.confirmarReserva();
            System.out.println();
        }

        // Comportamiento específico (versión sobrecargada, con nombre)
```

```
        System.out.println("=== Invocando comportamiento específico (sobrecarga) ===\n");
        ((ReservaClaseGrupal) reservas[0]).confirmarReserva("Profesora Ana Muñoz");
        System.out.println();
        ((ReservaEntrenamientoPersonal) reservas[1]).confirmarReserva("Entrenador Diego Vera
        System.out.println();
        ((ReservaCanchaLibre) reservas[2]).confirmarReserva("Encargado Pablo Rojas");
    }
}
```

> **Nota sobre el casteo en `Main`:** como `confirmarReserva(String)` es un método **nuevo** de cada subclase (sobrecarga) y no existe en `Reserva`, no puede invocarse a través de una referencia `Reserva`. Por eso se castea explícitamente a la subclase correspondiente. Una alternativa de diseño es declarar también `confirmarReserva(String nombreResponsable)` en `Reserva` (como método concreto o abstracto) para poder invocarlo polimórficamente sin casteo.

Salida de consola esperada

```
=== Invocando comportamiento base ===

[Reserva Clase Grupal]
Confirmando reserva...
-> Verificando cupos disponibles... OK

[Reserva Entrenamiento Personal]
Confirmando reserva...
-> Verificando disponibilidad del entrenador... OK

[Reserva Cancha Libre]
Confirmando reserva...
-> Verificando disponibilidad de la cancha... OK

=== Invocando comportamiento específico (sobrecarga) ===

[Reserva Clase Grupal]
Confirmando reserva...
-> Verificando cupos disponibles... OK
-> Reserva confirmada por Profesora Ana Muñoz

[Reserva Entrenamiento Personal]
Confirmando reserva...
-> Verificando disponibilidad del entrenador... OK
-> Reserva confirmada por Entrenador Diego Vera

[Reserva Cancha Libre]
Confirmando reserva...
-> Verificando disponibilidad de la cancha... OK
-> Reserva confirmada por Encargado Pablo Rojas
```

Ejercicio 5 — Extensión del sistema de reservas (principio Abierto/Cerrado)

Enunciado

La academia deportiva quiere lanzar una **cuarta modalidad**: reserva de **eventos especiales** (`ReservaEventoEspecial`), como torneos internos o celebraciones. Esta modalidad requiere una validación distinta: comprobar que el **salón multiuso esté disponible** y que exista **autorización de la administración**.

Se pide incorporar esta nueva modalidad **sin modificar** ninguna de las clases ya existentes (`Reserva`, `ReservaClaseGrupal`, `ReservaEntrenamientoPersonal`, `ReservaCanchaLibre`), demostrando así el principio de diseño **abierto/cerrado**.

Análisis de la solución

Gracias a que toda la lógica común (atributos `idReserva`, `horario`, `tipoReserva` y el constructor) vive encapsulada en `Reserva`, agregar una nueva modalidad consiste únicamente en **crear una subclase nueva** que siga el mismo contrato (sobrescribir `confirmarReserva()` y sobrecargarlo con el nombre del responsable). Ninguna clase existente se toca ni se recompila con cambios de lógica: el sistema **crece por extensión**, no por modificación del núcleo.

Resolución

```
// ReservaEventoEspecial.java  (única clase nueva — nada más se modifica)
public class ReservaEventoEspecial extends Reserva {

    public ReservaEventoEspecial(String idReserva, String horario) {
        super(idReserva, horario, "Evento especial");
    }

    // Sobrescritura: validación de salón y autorización
    @Override
    public void confirmarReserva() {
        System.out.println("[Reserva Evento Especial]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando disponibilidad del salón y autorización... OK");
    }

    // Sobrecarga: versión con nombre del responsable
    public void confirmarReserva(String nombreResponsable) {
        System.out.println("[Reserva Evento Especial]");
        System.out.println("Confirmando reserva...");
        System.out.println("-> Verificando disponibilidad del salón y autorización... OK");
        System.out.println("-> Reserva confirmada por " + nombreResponsable);
    }
}
```

```
// Main.java (fragmento agregado, el resto del Main del Ejercicio 4 no cambia)
Reserva nuevaReserva = new ReservaEventoEspecial("R-004", "20:00");
nuevaReserva.confirmarReserva();
((ReservaEventoEspecial) nuevaReserva).confirmarReserva("Coordinadora Marcela Fuentes");
```

Salida de consola esperada

```
[Reserva Evento Especial]
Confirmando reserva...
-> Verificando disponibilidad del salón y autorización... OK
[Reserva Evento Especial]
Confirmando reserva...
-> Verificando disponibilidad del salón y autorización... OK
-> Reserva confirmada por Coordinadora Marcela Fuentes
```

Ejercicio 6 — Sobrescritura: figuras geométricas y cálculo de área

Enunciado

Se requiere modelar figuras geométricas que calculen su propia área. Existen tres figuras: **Círculo**, **Rectángulo** y **Triángulo**, cada una con una fórmula distinta.

Se pide:

1. Crear una clase base `Figura` con un método `calcularArea()` que imprima un mensaje genérico.
2. Crear las subclases `Circulo` (atributo `radio`), `Rectangulo` (atributos `base` y `altura`) y `Triangulo` (atributos `base` y `altura`), cada una **sobrescribiendo** `calcularArea()` con su fórmula correspondiente e imprimiendo el resultado.
3. Desde `Main`, crear un arreglo `Figura[]` con una instancia de cada subclase y recorrerlo invocando `calcularArea()`.

Análisis de la solución

Cada figura encapsula sus propios atributos y su propia fórmula matemática. Gracias a la sobrescritura, el arreglo `Figura[]` puede contener objetos de distintas subclases y, al invocar `calcularArea()` sobre cada elemento, Java ejecuta automáticamente la versión correspondiente al tipo real del objeto —sin necesidad de preguntar "¿qué figura es?" mediante un `if`.

Resolución

```java
// Figura.java
public class Figura {
    public void calcularArea() {
        System.out.println("Área no definida para una figura genérica.");
    }
}
```

```java
// Circulo.java
public class Circulo extends Figura {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    @Override
    public void calcularArea() {
        double area = Math.PI * radio * radio;
        System.out.println("Círculo -> Área: " + String.format("%.2f", area));
    }
}
```

```java
// Rectangulo.java
public class Rectangulo extends Figura {
    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
```

```
        this.base = base;
        this.altura = altura;
    }

    @Override
    public void calcularArea() {
        double area = base * altura;
        System.out.println("Rectángulo -> Área: " + String.format("%.2f", area));
    }
}
```

```
// Triangulo.java
public class Triangulo extends Figura {
    private double base;
    private double altura;

    public Triangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override
    public void calcularArea() {
        double area = (base * altura) / 2;
        System.out.println("Triángulo -> Área: " + String.format("%.2f", area));
    }
}
```

```
// Main.java
public class Main {
    public static void main(String[] args) {
        Figura[] figuras = {
            new Circulo(4.0),
            new Rectangulo(5.0, 3.0),
            new Triangulo(6.0, 4.0)
        };

        for (Figura f : figuras) {
            f.calcularArea();
        }
    }
}
```

Salida de consola esperada

```
Círculo -> Área: 50.27
Rectángulo -> Área: 15.00
Triángulo -> Área: 12.00
```

Ejercicio 7 — Sobrecarga de métodos: clase Impresora

Enunciado

Se necesita una clase `Impresora` que ofrezca tres formas de imprimir un documento:

1. Sin parámetros: imprime un mensaje indicando que se envió el trabajo por defecto.
2. Recibiendo el **nombre del documento** (`String`): imprime indicando qué documento se envió.
3. Recibiendo el **nombre del documento** y el **número de copias** (`int`): imprime indicando el documento y cuántas copias se generarán.

Se pide implementar los tres métodos `imprimir()` sobrecargados y demostrarlos desde `Main`.

Análisis de la solución

Al igual que en el Ejercicio 2, aquí no hay herencia: los tres métodos conviven en la misma clase `Impresora`, comparten el nombre `imprimir`, y se distinguen únicamente por la **cantidad y el tipo de parámetros**. El compilador resuelve en tiempo de compilación cuál versión ejecutar según los argumentos entregados en cada llamada.

Resolución

```java
// Impresora.java
public class Impresora {

    // Versión 1: sin parámetros
    public void imprimir() {
        System.out.println("Enviando trabajo de impresión por defecto.");
    }

    // Versión 2: con nombre de documento
    public void imprimir(String nombreDocumento) {
        System.out.println("Imprimiendo documento: " + nombreDocumento);
    }

    // Versión 3: con nombre de documento y número de copias
    public void imprimir(String nombreDocumento, int copias) {
        System.out.println("Imprimiendo documento: " + nombreDocumento + " (" + copias + " c
    }
}
```

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        Impresora impresora = new Impresora();

        impresora.imprimir();
        impresora.imprimir("Informe_Final.pdf");
        impresora.imprimir("Informe_Final.pdf", 3);
    }
}
```

Salida de consola esperada

```
Enviando trabajo de impresión por defecto.
Imprimiendo documento: Informe_Final.pdf
Imprimiendo documento: Informe_Final.pdf (3 copias)
```

Ejercicio 8 — Polimorfismo combinado: sistema de préstamos de una biblioteca

Enunciado

Una biblioteca gestiona tres tipos de materiales: **Libro**, **Revista** y **DVD**, cada uno con una multa distinta por día de atraso en la devolución:

- `Libro`: multa de $200 por día de atraso.
- `Revista`: multa de $100 por día de atraso.
- `DVD`: multa de $500 por día de atraso.

Se pide diseñar el sistema aplicando polimorfismo, en dos fases:

Fase 1 — Comportamiento diferenciado (sobrescritura): - Crear una clase base `Material` con los atributos encapsulados `codigo` y `titulo`, y un método `calcularMulta(int diasAtraso)` con un mensaje genérico. - Crear las subclases `Libro`, `Revista` y `DVD`, cada una **sobrescribiendo** `calcularMulta(int diasAtraso)` con su propia tarifa.

Fase 2 — Flexibilidad de cálculo (sobrecarga): - En la clase `Material`, agregar una **segunda versión** `calcularMulta()` (sin parámetros) que informe que no hay atraso registrado, de modo que `Material` (y por herencia sus subclases) ofrezca ambas versiones del método.

Prueba de integración: desde `Main`, instanciar un objeto de cada subclase e invocar ambas versiones del método.

Análisis de la solución

Este ejercicio combina, en la **misma jerarquía**, los dos mecanismos de polimorfismo:

- **Sobrescritura:** `calcularMulta(int diasAtraso)` se redefine en cada subclase con la misma firma, pero aplicando una tarifa distinta.
- **Sobrecarga:** `calcularMulta()` (sin parámetros) es una firma distinta dentro de la **misma clase base** `Material`, y al no ser redefinida en las subclases, se hereda tal cual — demostrando que sobrecarga y sobrescritura son mecanismos independientes que pueden convivir en una misma jerarquía.

Resolución

```java
// Material.java
public class Material {
    private String codigo;
    private String titulo;

    public Material(String codigo, String titulo) {
        this.codigo = codigo;
        this.titulo = titulo;
    }

    public String getCodigo() {
        return codigo;
    }

    public String getTitulo() {
        return titulo;
    }
```

```
    // Versión base (con parámetro) — será sobrescrita en cada subclase
    public void calcularMulta(int diasAtraso) {
        System.out.println(titulo + ": multa genérica no definida.");
    }

    // Sobrecarga en la clase base: sin parámetros
    public void calcularMulta() {
        System.out.println(titulo + ": no hay atraso registrado, multa $0.");
    }
}
```

```
// Libro.java
public class Libro extends Material {
    public Libro(String codigo, String titulo) {
        super(codigo, titulo);
    }

    @Override
    public void calcularMulta(int diasAtraso) {
        int multa = diasAtraso * 200;
        System.out.println(getTitulo() + " (Libro): multa de $" + multa + " por " + diasAtra
    }
}
```

```
// Revista.java
public class Revista extends Material {
    public Revista(String codigo, String titulo) {
        super(codigo, titulo);
    }

    @Override
    public void calcularMulta(int diasAtraso) {
        int multa = diasAtraso * 100;
        System.out.println(getTitulo() + " (Revista): multa de $" + multa + " por " + diasAt
    }
}
```

```
// DVD.java
public class DVD extends Material {
    public DVD(String codigo, String titulo) {
        super(codigo, titulo);
    }

    @Override
    public void calcularMulta(int diasAtraso) {
        int multa = diasAtraso * 500;
        System.out.println(getTitulo() + " (DVD): multa de $" + multa + " por " + diasAtraso
    }
}
```

```
// Main.java
public class Main {
    public static void main(String[] args) {
        Material[] materiales = {
            new Libro("L-01", "Cien años de soledad"),
            new Revista("R-01", "National Geographic"),
```

```
            new DVD("D-01", "El viaje de Chihiro")
        };

        System.out.println("=== Multas con atraso (sobrescritura) ===");
        for (Material m : materiales) {
            m.calcularMulta(3);
        }

        System.out.println("\n=== Multas sin atraso (sobrecarga heredada) ===");
        for (Material m : materiales) {
            m.calcularMulta();
        }
    }
}
```

Salida de consola esperada

```
=== Multas con atraso (sobrescritura) ===
Cien años de soledad (Libro): multa de $600 por 3 día(s) de atraso.
National Geographic (Revista): multa de $300 por 3 día(s) de atraso.
El viaje de Chihiro (DVD): multa de $1500 por 3 día(s) de atraso.

=== Multas sin atraso (sobrecarga heredada) ===
Cien años de soledad: no hay atraso registrado, multa $0.
National Geographic: no hay atraso registrado, multa $0.
El viaje de Chihiro: no hay atraso registrado, multa $0.
```

Ejercicio 9 — Análisis de código: identificar sobrescritura y sobrecarga

Enunciado

Analiza el siguiente fragmento de código de un sistema de notificaciones y responde las preguntas planteadas debajo:

```java
public class Notificacion {
    public void enviar() {
        System.out.println("Enviando notificación genérica.");
    }

    public void enviar(String destinatario) {
        System.out.println("Enviando notificación a " + destinatario);
    }
}

public class NotificacionUrgente extends Notificacion {
    @Override
    public void enviar() {
        System.out.println("Enviando notificación URGENTE.");
    }
}
```

Preguntas:

1. ¿Cuáles métodos de este código son un ejemplo de **sobrecarga** y por qué?
2. ¿Cuál método es un ejemplo de **sobrescritura** y por qué?
3. Si se ejecuta `new NotificacionUrgente().enviar("Ana")`, ¿qué mensaje se imprime? Justifica.
4. ¿Es válido escribir `new NotificacionUrgente().enviar(3)`? Justifica tu respuesta.

Resolución

1. Sobrecarga: los dos métodos `enviar()` y `enviar(String destinatario)` **dentro de la clase `Notificacion`** son un ejemplo de sobrecarga, porque comparten el mismo nombre pero tienen **distinta lista de parámetros** (uno no recibe nada, el otro recibe un `String`). El compilador decide cuál invocar según los argumentos entregados.

2. Sobrescritura: el método `enviar()` en `NotificacionUrgente` es un ejemplo de sobrescritura, porque **redefine** con la **misma firma** (sin parámetros) el método `enviar()` heredado de `Notificacion`, cambiando su comportamiento. Está marcado explícitamente con `@Override`.

3. Se imprime:

```
Enviando notificación a Ana
```

Justificación: `NotificacionUrgente` **no sobrescribe** la versión `enviar(String destinatario)`, por lo tanto hereda tal cual la implementación de `Notificacion`. Al llamar `enviar("Ana")`, Java busca una versión con un parámetro `String` — la encuentra heredada sin cambios — y la ejecuta.

4. No es válido. No existe ninguna versión de `enviar()` que reciba un `int` como parámetro (ni en `Notificacion` ni en `NotificacionUrgente`), por lo que el código **no compila**. Java no realiza conversión automática de `int` a `String` en la resolución de sobrecarga.

Ejercicio 10 — Extensión del sistema de biblioteca (principio Abierto/Cerrado)

Enunciado

La biblioteca del Ejercicio 8 quiere incorporar un nuevo tipo de material: **Audiolibro** (`Audiolibro`), con una multa de **$150 por día de atraso**.

Se pide incorporar este nuevo material **sin modificar** ninguna de las clases ya existentes (`Material`, `Libro`, `Revista`, `DVD`), demostrando el principio de diseño **abierto/cerrado**.

Análisis de la solución

Como toda la lógica común (`codigo`, `titulo`, constructor y la sobrecarga `calcularMulta()` sin atraso) vive en `Material`, agregar el nuevo material solo requiere **crear una subclase** que herede de `Material` y sobrescriba `calcularMulta(int diasAtraso)` con su propia tarifa. Ninguna clase existente necesita tocarse: el sistema **se extiende**, no se modifica.

Resolución

```java
// Audiolibro.java  (única clase nueva — nada más se modifica)
public class Audiolibro extends Material {
    public Audiolibro(String codigo, String titulo) {
        super(codigo, titulo);
    }

    @Override
    public void calcularMulta(int diasAtraso) {
        int multa = diasAtraso * 150;
        System.out.println(getTitulo() + " (Audiolibro): multa de $" + multa + " por " + dia
    }
}
```

```java
// Main.java (fragmento agregado, el resto del Main del Ejercicio 8 no cambia)
Material nuevoMaterial = new Audiolibro("A-01", "Sapiens");
nuevoMaterial.calcularMulta(2);
nuevoMaterial.calcularMulta();
```

Salida de consola esperada

```
Sapiens (Audiolibro): multa de $300 por 2 día(s) de atraso.
Sapiens: no hay atraso registrado, multa $0.
```

Conclusión del repositorio

Los diez ejercicios evidencian cómo el **polimorfismo** —por **sobrescritura** entre clase base y subclases, y por **sobrecarga** dentro de una misma clase o jerarquía— permite transformar sistemas condicionales rígidos (`if/else` o `switch` por tipo) en **arquitecturas escalables**, donde agregar un nuevo comportamiento (un nuevo tipo de empleado, de figura, de material o de modalidad de reserva) no exige modificar el código ya probado y en producción.